Production Checkpoint Report

FireSVM · MobileNet-SSD · Thermo-X3D T5v2

Smart Thermal System for Patient Safety Monitoring

Guy Chen Yaniv Blau
Roy Lieberman
Supervisor: Or Zilberberg Advisor: Dr. Oshrit Hoffer Afeka College of Engineering

July 4, 2026

Abstract

This report documents the three model checkpoints deployed in the live demonstration system: the **FireSVM** fire detector, the **MobileNet-SSD** person detector, and the **Thermo-X3D T5v2** contact (touch) detector. For each model we describe the architecture, inputs and outputs, feature extraction, loss and optimizer, the heuristics that surround the network, benchmark accuracy (accuracy/precision/recall/F_1/FAR), and measured CPU runtime with estimates for the Raspberry Pi 5 target. All sensing is thermal-only (Waveshare 80×62 module, 8 Hz) — no optical imagery is ever captured.

1 System context

Three thermal cameras observe the room. Each 8 Hz tick produces three raw 62×80 frames in °C. Two preprocessing views feed the models (an empirically-validated split — see §4.4):

- **Raw frames** go to the per-camera detectors (FireSVM, MobileNet-SSD). Raw input transfers across rooms without recalibration.
- **Tateno residuals** go to Thermo-X3D (contact).

Task	Checkpoint file	Format	Size
Fire	`fire_svm_detector/_default.thalg`	sklearn pickle	64 KB
Person	`mobilenet_ssd_detector/Waveshare_26984_raw.onnx`	ONNX	0.27 MB
Contact	`thermo_x3d_detector/Waveshare_26984_T5_v2.ftz.onnx` (+meta)	ONNX	1.6 MB

Table 1: Deployed checkpoints. All are committed on the `demo-linux` branch; total ≈2 MB. The two neural networks run via `onnxruntime` — the deployment is torch-free.

1.1 Preprocessing: the Tateno pipeline, and raw vs. residual

How it works. The Tateno pipeline (Engineering Report §4.4.1) turns a raw thermal frame into a *motion/novelty residual* in three stages:

1. **Gaussian smoothing.** Convolve the frame with a Gaussian kernel to suppress the per-pixel sensor noise (the Waveshare module has a ∼0.7°C noise floor). The kernel width is set in *physical* units — a 5 cm smoothing scale converted to pixels via the sensor's angular resolution at the working distance — so the same setting behaves consistently as geometry changes.
2. **Background subtraction.** Subtract a per-pixel background image $B(x, y)$: $d = \text{smooth}(I) - B$. B is the temperature each pixel shows when the scene is "empty/static". A pixel that matches the background cancels to zero; a warm body that was not there stands out.
3. **Rectified L_1 residual.** Take the absolute value, $r = |d|$. Both hotter-than-background (a person) and colder-than-background (a door opened to a cool corridor) become positive novelty; the output is a non-negative "how different from normal is this pixel" map.

Where the background comes from. The background is not a fixed calibration image (that would violate the no-per-room-calibration constraint). At deployment it is the *session-local rolling 25th-percentile* of the last ∼30 s of frames, refreshed continuously (∼2 s warm-up). The p25 (rather than the mean or median) is deliberate: over a short window a person who stands still would leak into a mean/median background and then cancel themselves out; the 25th percentile tracks the "cool, empty" end of each pixel's recent history, so stationary people still register. Empirically p25 matched an ideal empty-room background at home *and* transferred to a new room where mean/median failed.

When we use which. This choice was settled by a preprocessing ablation (§4.4) and is not uniform across models:

- **Raw frames** → FireSVM and MobileNet-SSD. Fire is an *absolute*-temperature phenomenon (a flame is hot regardless of the background), and the SSD trained on raw frames both scored higher and transferred across rooms; background subtraction actively hurt it.
- **Tateno residual** → Thermo-X3D. Contact is about *relative* body positions and motion. On raw frames warm bodies barely exceed a warm room and blob structure collapses; the residual makes the two interacting bodies the dominant signal. The contact model is also trained on residuals, so it sees at inference exactly what it saw in training.

The one-line rule: *raw for the detectors that ask "how hot / who is here", residual for the detector that asks "how are the bodies moving relative to normal".*

2 FireSVM — fire detection

2.1 What is an SVM?

Our classifier sees each frame as one point $x \in \mathbb{R}^6$ (the six statistics of the hottest blob, z-scored) with label $y{=}+1$ for fire frames and $y{=}-1$ for everything else. A linear SVM seeks a hyperplane $w^\top x + b = 0$ separating the two classes with *maximum margin*: among all separating hyperplanes, pick the one with the widest empty corridor between the boundary and the closest point of either class. Requiring $y_i(w^\top x_i + b) \geq 1$ for all training points makes the corridor half-width $1/\|w\|$, so maximizing the margin is minimizing $\|w\|^2$. Wide margins generalize: the boundary is fixed by the hardest points — the *support vectors* — and ignores everything comfortably far away.

Soft margin and the meaning of C. The hard-margin problem assumes a perfect corridor exists. For our data it does not: the fire and non-fire clouds *overlap in feature space* — a cigarette ember at 3 m is a 2-pixel ∼50°C blob whose six statistics nearly coincide with a hot corner of the heater (labeled SAFE), and a few frames are mislabeled outright. With overlapping classes the constraints $y_i(w^\top x_i + b) \geq 1\ \forall i$ are *infeasible* — the hard-margin SVM has no solution. The fix is to let points violate the corridor but charge for it, via slack variables $\xi_i \geq 0$ measuring how badly each point violates:

$$\min_{w,b,\xi}\ \tfrac{1}{2}\|w\|^2 + C\sum_i \xi_i \qquad \text{s.t.} \quad y_i(w^\top x_i + b)\ \geq\ 1 - \xi_i.$$

$\xi_i{=}0$: outside the corridor, correct side (free). $0{<}\xi_i{<}1$: correct side but inside the corridor. $\xi_i{>}1$: misclassified. C **is the price per unit of violation**, arbitrating margin width against training errors: $C \to \infty$ recovers the hard margin (the boundary contorts around every noisy point — overfitting); $C \to 0$ makes violations free (the optimizer ignores the data — underfitting).

Why C is a regularizer. Rewriting the same optimization in loss-plus-penalty form makes this explicit:

$$\min_{w,b}\ \sum_i \max(0,\, 1 - y_i(w^\top x_i + b))\ +\ \tfrac{1}{2C}\|w\|^2,$$

i.e. *hinge loss plus L_2 weight decay, with C the inverse of the weight-decay strength* — exactly the capacity-control role weight decay plays in a neural network. Our C=1 (a moderate default) leaves 874 of 6,497 training points as support vectors (∼13% on or inside the corridor), which quantifies the genuine class overlap.

The dual and the kernel trick. A single hyperplane over the six raw features cannot express interactions such as "fire = very hot *and* small, or moderately hot *and* heavy-tailed". The classical remedy is a feature map $\phi(x)$ that appends products and powers, then separates linearly in the bigger space — but a rich enough ϕ is enormous (for the kernel below, infinite-dimensional), so computing it explicitly is intractable. The escape is the Lagrangian *dual* of the soft-margin problem:

$$\max_{\alpha} \sum_i \alpha_i - \tfrac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle \qquad \text{s.t.} \quad 0 \le \alpha_i \le C, \quad \textstyle\sum_i \alpha_i y_i = 0.$$

The data enters *only through inner products* $\langle x_i, x_j \rangle$ — at training and at prediction. Therefore we never need $\phi(x)$ itself, only $\langle \phi(x), \phi(x') \rangle$, which we compute directly with a **kernel** $k(x, x') = \langle \phi(x), \phi(x') \rangle$. Mercer's theorem guarantees that every symmetric positive semi-definite k is the inner product of *some* feature map, so in practice one chooses k and never constructs ϕ. (Note where C reappears: as the box constraint $\alpha_i \le C$, capping the influence any single training point may exert — the dual view of the same regularization.)

The RBF kernel. RBF = radial basis function — "radial" because it depends only on the distance between the two points:

$$k(x, x') = \exp(-\gamma \|x - x'\|^2).$$

Three complementary readings: (i) a *similarity score* that is 1 for identical blob statistics and decays smoothly with distance, with γ setting the length scale — `scale` gives $\gamma = 1/(d \cdot \mathrm{Var}(X)) \approx 1/6$ on our z-scored features, so differences of about one standard deviation per feature are what count; (ii) an implicit ϕ: Taylor-expanding the exponential shows the RBF feature space contains *all polynomial degrees at once* (infinite-dimensional), making the RBF-SVM a universal approximator, with γ controlling capacity ($\gamma \to \infty$ memorizes each point, $\gamma \to 0$ degenerates toward linear); (iii) the prediction rule below.

Prediction and Platt scaling. The trained classifier is

$$\hat{y} = \mathrm{sign}\Big(\textstyle\sum_{i \in \mathrm{SV}} \alpha_i \, y_i \, k(x_i, x) + b \Big),$$

a *distance-weighted vote of memorized prototypes*: the 874 support vectors stored in the checkpoint *are* the model. A new blob computes its similarity to each prototype; fire prototypes pull the score up in their neighborhood, SAFE prototypes pull it down, each with learned strength α_i (bounded by C). The decision surface is a smooth landscape of Gaussian bumps centered on the support vectors. The raw score $f(x)$ measures distance to the boundary — useful for ranking but not calibrated. *Platt scaling* fits a one-dimensional logistic regression $P(\text{fire} \mid x) = \sigma(a\, f(x) + c)$ on cross-validated decision values (two parameters, fitted after the SVM); this is the confidence reported in every `FireAlert`.

Terminology: SVM, SVC, SMO. *SVM* is the family of max-margin kernel methods; *SVC* ("support vector *classification*") is scikit-learn's implementation of the soft-margin kernel SVM classifier — "RBF-SVC" simply means SVC configured with the RBF kernel, exactly the model above. *SMO* (sequential minimal optimization) is the algorithm that solves the dual: the dual is a quadratic program over α with one equality constraint, and SMO exploits the fact that the smallest feasible update is a *pair* of coordinates (α_i, α_j) — chosen by which points currently violate the optimality conditions — for which the constrained optimum has a closed form. Iterating over such

pairs solves the QP without any generic solver; convergence to the global optimum is guaranteed because the dual is convex. This is why FireSVM has no optimizer, learning rate, or epoch schedule to report: training is a convex problem solved exactly, and the only capacity knobs are C and γ.

2.2 What is special about our FireSVM

FireSVM is not applied to pixels. A classical segmentation stage proposes a region, and the SVM classifies six scalar statistics of that region — which makes the model *resolution-invariant* (the same checkpoint works on 32×24 and 80×62 sensors) and extremely fast.

Pipeline (per raw frame).

1. **Region proposal — Otsu segmentation.** The frame is histogrammed (256 bins); Otsu's criterion selects the threshold maximizing between-class variance; the binary mask is shaped by one 3×3 erosion and two dilations.
2. **Feature extraction.** Connected components are extracted and the *hottest blob* is selected. Six features describe it: max temperature, mean, standard deviation, area, skewness and kurtosis of the blob's pixel distribution. (Fire blobs are small, extremely hot and heavy-tailed; people are large, near 36°C and symmetric.)
3. **Standardization.** Features are z-scored with the μ, σ learned at fit time.
4. **Classification.** RBF-SVC (C=1, γ=scale, 874 support vectors) outputs SAFE or ACTIVE_COMBUSTION with a Platt confidence.

Skewness and kurtosis. Four of the six features (max, mean, std, area) are intuitive. The last two describe the *shape* of the blob's pixel-temperature distribution — its third and fourth standardized moments. For blob pixels $\{t_i\}$ with mean μ and std σ:

$$\text{skewness} = \frac{1}{n}\sum_i \Big(\frac{t_i-\mu}{\sigma}\Big)^3, \qquad \text{kurtosis} = \frac{1}{n}\sum_i \Big(\frac{t_i-\mu}{\sigma}\Big)^4 - 3.$$

Skewness measures asymmetry: 0 for a symmetric distribution, positive when a long tail points toward high temperatures. A flame has a few very hot pixels trailing off from a cooler edge — strongly positive skew — whereas a person's torso is roughly symmetric around body temperature. *Kurtosis* (excess, hence the -3 so a Gaussian scores 0) measures "tailedness"/peakedness: high when the distribution has a sharp peak and heavy tails — extreme outliers relative to the spread. Fire, which flickers, produces exactly such heavy-tailed pixel distributions; a uniform warm body does not. These two moments are what let six scalars separate "small hot flickering thing" from "large steady warm thing" without ever looking at the pixels' spatial layout.

Input/output. In: one raw 62×80 float32 frame (°C). Out: a `FireAlert` — level, Platt confidence, and the hottest blob's features including its bounding box (drawn in the demo UI).

Training. Frames with a YOLO class-0 (fire) box are positives; all other labeled frames are SAFE. Stratified sequential 70/30 split per (scene, camera, class). No gradient training — the SVC solves its convex dual (SMO); there is no optimizer/epoch schedule, and regularization is the C soft margin.

Runtime note. Profiling showed 88% of inference time was *scipy call overhead* for skewness/kurtosis (∼13 blobs × 2 calls per frame). Replacing them with an algebraically identical numpy implementation (verified bit-identical on 1,395 real blobs) cut predict from 8.8 to **1.19 ms/frame** with zero behavioral change.

	Accuracy	Precision	Recall	F_1	FAR	TP	TN	FP	FN
FireSVM (test)	96.4%	99.7%	75.5%	85.9%	0.04%	314	2410	1	102

Table 2: FireSVM benchmark — 2,827 held-out test frames (416 fire-positive) from the 70/30 protocol; 6,497 training frames. FAR is the false-alarm rate over fire-free frames. The near-zero FAR is the design priority for a ward: the detector essentially never cries wolf, at the cost of missing low-contrast ignition frames (small cigarette embers at distance).

3 MobileNet-SSD — person detection

3.1 Architecture

A single-stage anchor-based detector (SSD) on a MobileNet-style backbone, sized for our tiny native input — we do *not* upscale the thermal frame to 300×300 like standard SSD; the network runs on $1 \times 62 \times 80$ directly.

Why depthwise-separable convolutions. A standard convolution mixes space and channels in one shot: a 3×3 conv from C_{in} to C_{out} channels costs $3{\cdot}3{\cdot}C_{in}{\cdot}C_{out}$ multiply–accumulates per output pixel. A *depthwise-separable* block factors this into two cheaper steps: (i) a **depthwise** 3×3 conv that filters each input channel independently (spatial mixing only, $3{\cdot}3{\cdot}C_{in}$ MACs), then (ii) a **pointwise** 1×1 conv that recombines channels (channel mixing only, $C_{in}{\cdot}C_{out}$ MACs); each is followed by BatchNorm+ReLU. The cost ratio is $\frac{1}{C_{out}} + \frac{1}{9}$ — roughly 8–9× fewer operations for the channel counts we use — at a small accuracy cost. On a CPU-only edge device that factorization is what keeps the detector at a few milliseconds per frame.

Stage	Operator	Stride	Output ($C \times H \times W$)
Input			$1 \times 62 \times 80$
Stem	Conv 3×3 + BN + ReLU	2	$16 \times 31 \times 40$
Block 1	DW-separable	1	$32 \times 31 \times 40$
Block 2	DW-separable	2	$64 \times 15 \times 20$ ($\rightarrow$ **F1**)
Block 3	DW-separable	1	$64 \times 15 \times 20$
Block 4	DW-separable	2	$128 \times 7 \times 10$ ($\rightarrow$ **F2**)
Block 5	DW-separable	1	$128 \times 7 \times 10$
Head (F1)	Conv 3×3 $\rightarrow K(4{+}2)$ ch	1	box $12 \times 15 \times 20$, cls $6 \times 15 \times 20$
Head (F2)	Conv 3×3 $\rightarrow K(4{+}2)$ ch	1	box $12 \times 7 \times 10$, cls $6 \times 7 \times 10$

Table 3: MicroMobileNet-SSD, Waveshare configuration. Two detection scales: F1 (fine, near people) and F2 (coarse, large/close people). K=3 anchors per cell $\Rightarrow 15{\cdot}20{\cdot}3 + 7{\cdot}10{\cdot}3 = 1110$ anchors total.

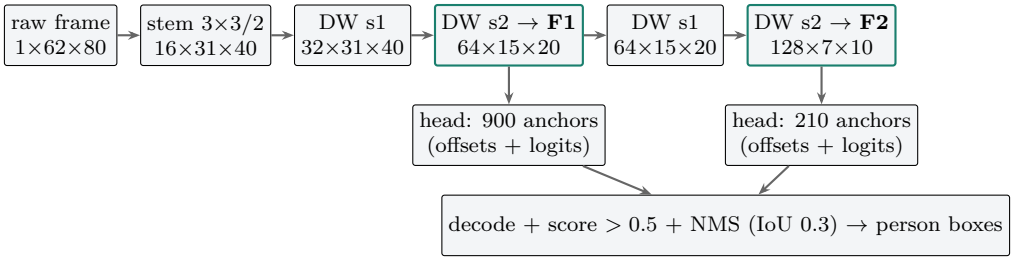

Figure 1: Data flow. (Block 5, a stride-1 refinement after F2, omitted for clarity.)

F1 and F2 — the two detection scales. **F1** and **F2** are the two *feature maps* the backbone taps for detection — not to be confused with the F_1 *score* metric. F1 is the earlier, higher-resolution map ($64 \times 15 \times 20$); F2 is the deeper, coarser map ($128 \times 7 \times 10$). SSD predicts boxes at both because

a single scale cannot cover the range of apparent person sizes in the room: F1's small cells and small anchors catch people who are far away or small in the frame, while F2's large cells and large anchors catch people who are close and fill much of the view. Each map's cells carry their own anchors, and their predictions are concatenated before decoding — a cheap way to get scale invariance without an image pyramid.

Anchors. Each feature-map cell holds K=3 anchor boxes with aspect ratios {1:1, 1:2, 1:3} (square, tall, very tall — person-shaped) and base widths of 10 px (F1) and 16 px (F2) in input coordinates. An *anchor* is a fixed reference box at a cell; the network predicts a small offset from it rather than absolute coordinates, which makes regression easier and stable.

Input/output. In: one raw frame, per-frame z-scored $(x-\mu)/(\sigma+10^{-6})$. Out: for each of the 1110 anchors, 4 box offsets and 2 class logits. Offsets are decoded against the anchor (cx, cy scaled by variance 0.1; w, h by $e^{0.2\,d}$), softmax scores are thresholded at 0.5, and greedy non-maximum suppression (IoU 0.3) yields the final `(x,y,w,h,score)` person boxes.

3.2 Loss, optimizer, heuristics

Loss is the standard SSD multi-task objective. Anchors are matched to ground truth by IoU (positive ≥ 0.5, negative < 0.4, in-between ignored):

$$\mathcal{L} \;=\; \frac{1}{N_{pos}}\Big(\underbrace{\sum \mathrm{CE}(\mathrm{cls})}_{\text{pos + hard negatives}} + \lambda \underbrace{\sum \mathrm{SmoothL1}(\mathrm{loc})}_{\text{positives only}}\Big), \qquad \lambda = 1.$$

The loss is "multi-task" because it trains two heads at once, one term each:

- **CE — cross-entropy (classification).** For each considered anchor the class head emits two logits (person vs. background); softmax turns them into probabilities and cross-entropy $-\log p(\text{true class})$ penalizes the model in proportion to how confidently it was wrong. This teaches *which* anchors contain a person.
- **SmoothL1 — localization (box regression).** For anchors matched to a real person, the box head predicts four offsets (center shift and log-size change) from the anchor to the true box; SmoothL1 (Huber) loss scores the error. It is quadratic for small errors (precise fitting near the target) but linear for large ones ($|e| > 1$), so a few badly-placed anchors early in training cannot produce exploding gradients the way plain squared error would. This teaches *where* the box goes, and runs on positives only — there is no box to regress for background.

Both sums are divided by N_{pos} (the number of matched anchors) so the loss scale does not depend on how many people are in the frame.

Hard-negative mining keeps only the 3 highest-loss background anchors per positive (3:1), preventing the 1110-to-few anchor imbalance from drowning the signal (almost every anchor is background, so without this the classifier would trivially predict "background" everywhere). **Optimizer:** Adam, lr 10^{-3}, weight decay 10^{-4} (the only explicit regularizer besides BatchNorm), 50 epochs. **Augmentation:** random horizontal flips and small random translations (zero-padded), boxes transformed accordingly.

NMS — non-maximum suppression (at inference). Because neighboring anchors and both feature maps fire on the same person, one body produces many overlapping boxes. NMS collapses them to one: sort the surviving boxes by score, keep the highest, discard every remaining box that overlaps it by more than an IoU threshold (0.3 here), and repeat on what is left. The result is one box per person — the `(x,y,w,h,score)` detections drawn in the demo. It is a classical greedy geometric post-process, not learned.

3.3 Results

Benchmarked on the Waveshare dataset with an 80/10/10 timeline split (frame-level presence; box quality as mean IoU of matched detections):

Detector (test)	Accuracy	Precision	Recall	F_1	FAR	mean IoU
AdaptiveThreshold	76.6%	93.4%	80.7%	86.6%	83.0%	0.565
HOG+SVM	79.8%	92.7%	85.1%	88.7%	98.1%	0.585
MobileNet-SSD	**95.8%**	**99.1%**	**96.4%**	**97.7%**	**13.2%**	**0.673**

Table 4: Person detection, 825 test frames (772 with people). FAR is computed over the 53 person-free test frames (7 false alarms). The deployed checkpoint is the *raw-input* variant of the same architecture and training recipe: a preprocessing ablation showed the SSD performs best on raw frames (and transfers across rooms), while background-subtracted residuals belong to the contact model.

For deployment the network was exported to ONNX with the anchor decode and NMS re-implemented in numpy; parity vs. the PyTorch path was verified at 7.6×10^{-6} max box/score difference over 330 boxes.

4 Thermo-X3D T5v2 — contact (touch) detection

4.1 Architecture

Contact between people is a *spatio-temporal* event — two warm blobs approaching, merging, and interacting over seconds. Thermo-X3D is a small video CNN inspired by the X3D family: it processes a sliding window of T=5 consecutive residual frames from all three cameras and outputs one contact probability. It is *pixel-based end-to-end*: no bounding boxes, no cross-camera geometry, and crucially **no homography or per-room calibration** (a hard product constraint — 800 rooms).

Every convolution is **(2+1)D-factorized**: a spatial 1×3×3 kernel followed by a temporal 3×1×1 kernel. Factorization halves parameters and FLOPs versus full 3^3 kernels, doubles the non-linearities, and — practically — means every op is effectively a 2-D convolution, which CPU inference libraries optimize well.

Stage	Operator	Spatial stride	Output ($C \times T \times H \times W$)
Input (per camera)			$1 \times 5 \times 62 \times 80$
Stem	Conv 1×3×3 + BN + ReLU	1	$24 \times 5 \times 62 \times 80$
ResBlock 1	(2+1)D + attention + skip	1	$24 \times 5 \times 62 \times 80$
ResBlock 2	(2+1)D + attention + skip	2	$48 \times 5 \times 31 \times 40$
ResBlock 3	(2+1)D + attention + skip	2	$96 \times 5 \times 16 \times 20$
Pool + proj	global avg pool, Linear 96→128		128
Fusion	concat 3 camera streams		384
Head	Linear 384→64, ReLU, Linear 64→2		2 logits

Table 5: Thermo-X3D T5v2. Three *independent* per-camera encoder streams (late fusion). 382,562 parameters total.

Each residual block is: spatial conv → BN/ReLU → temporal conv → BN/ReLU → **thermal attention** → add skip (1×1×1 conv when shape changes) → ReLU. The attention module is CBAM-style, adapted to thermal statistics where *maxima* matter (hot spots):

- *Channel attention:* global average- **and** max-pool over (T, H, W), each through a shared 2-layer MLP (reduction 4), summed, sigmoid-gated per channel.
- *Spatial attention:* channel-wise mean and max maps concatenated, convolved 1×7×7, sigmoid-gated per location.

Why attention here is cheap (and why not Mamba). This is **CBAM-style convolutional attention**, not the self-attention of a Transformer — the distinction matters for CPU cost. Transformer/self-attention builds an $N \times N$ token-to-token similarity matrix, so its cost grows *quadratically* with sequence length; that is the expense people mean when they say "attention is heavy", and it is what state-space models (Mamba) exist to avoid by replacing the quadratic mixer with a linear-time recurrence. Our attention has none of that structure. Channel attention is two global pools plus a tiny MLP on a length-C vector ($O(C)$); spatial attention is a mean, a max, and one 7×7 convolution over the feature map ($O(HW)$). There is no token–token product and no quadratic term — the module adds a couple of percent to the layer's FLOPs. Mamba/state-space blocks solve a problem we do not have (long-sequence quadratic attention over T=5 frames would be pointless) and would add a recurrent scan that ONNX/onnxruntime optimize far less well than the plain convolutions and pools we already use. The gate simply lets the network emphasize the hot, moving regions (channel and spatial hot spots) that signal contact — worth its small cost, measured in the runtime table.

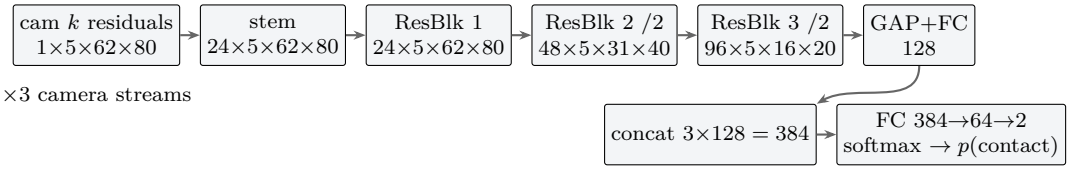

Figure 2: One of three per-camera streams; features fuse late.

Why global *average* pool at the head. Two different poolings appear in this network, for two different jobs, and it is worth separating them. *Downsampling* inside the backbone is done by *strided convolution*, not max-pool — modern practice, since a learned stride keeps more information than a hard max and exports cleanly to ONNX. The final collapse of the $96 \times 5 \times 16 \times 20$ tensor to a 128-d descriptor uses **global average pooling**, which is the standard head in modern classification backbones (ResNet, X3D, etc.), not global max. The reason is what the two poolings assume about the signal. Global *max* keeps only the single strongest activation per channel — ideal when the class is defined by one localized peak. Contact is the opposite: it is a *distributed, holistic* event — two bodies, the region where they meet, their joint motion across the frame — so averaging aggregates evidence over the whole scene and is robust to any single hot pixel, whereas max would throw away the spatial extent that distinguishes "two people touching" from "one hot spot". Note we do *not* discard max entirely: the attention module (above) uses average *and* max pooling together, precisely because peak detection (hot spots) helps *gating* even though averaging is the right *final* aggregation.

Inputs/outputs. In: for each camera, the Tateno residual against the rolling p25 background, globally z-scored (μ=0.868, σ=0.786, statistics of the training pixels); a rolling buffer holds the last T=5 frames per camera, so each 8 Hz tick re-evaluates the network on a $(3, 5, 62, 80)$ volume spanning 0.625 s. Out: softmax contact probability. **Decision heuristics:** alarm threshold 0.35 (selected by validation sweep), evaluated every tick; the pipeline supports N-frame persistence (deployed as benchmarked: N=1).

4.2 Loss, optimizer, training data

Loss: cross-entropy with *inverse-frequency class weights* (positives are ∼35% of frames; weights $n/(2n_{neg}), n/(2n_{pos})$). **Optimizer:** Adam, lr 10^{-3}, weight decay 10^{-4}, 10 epochs, batch 8, over all T-frame windows of contiguous training runs. **Data:** 57 scenes / 30,279 frames / 10,516 positive — the home recordings (re-annotated after a label audit) plus the 06-30 ward-archive segments; per-scene timeline split 60/15/25; the human-labeled `2men_clash` scene is excluded entirely as a

held-out benchmark. Training uses the same p25-background residuals as deployment (v2's key fix — the model trains on exactly what it sees at runtime).

4.3 Results

	Accuracy	Precision	Recall	F_1	FAR
In-domain test (7,592 frames, th 0.35)	91.3%	85.6%	90.1%	87.8%	8.0%
Held-out `2men_clash`, frame-exact	—	23.1%	62.2%	33.7%	54.4%
Held-out `2men_clash`, ±2 frames	—	28.1%	73.7%	40.7%	51.3%

Table 6: Thermo-X3D T5v2. The held-out gap is a *label-granularity* artifact, not pure model error: much of the training corpus carries episode-level interval labels (contact marked continuously over multi-minute episodes), teaching episode semantics that frame-exact scoring penalizes. At the *episode* level the model detects 5/5 contact episodes in the held-out scene with 12.8 s of total false-positive time. Frame-exact tightening is the main open item for the next retrain (interval-core training or threshold recalibration).

ONNX backend verification. The deployed ONNX checkpoint was re-scored on the full protocol: fp32 ONNX reproduces PyTorch *exactly* (F_1 87.8% / 33.7%). An int8-quantized variant was also produced (test F_1 86.6%, FAR 8.0→5.2%) but is *slower* than fp32 on x86 (no optimized int8 3-D conv path in onnxruntime) and is kept only as a Pi-side candidate for future testing.

4.4 The classical alternative — "config-D" rule

Alongside the network we maintained a hand-crafted contact rule, the best of a long ablation series (**config-D**): run MobileNet-SSD on *raw* frames; segment warm blobs on the Tateno *residual*; fire when *exactly two* person boxes fall inside one connected warm component (the two-body merge test — rejects person+object and 3-person crowds); then clean the per-frame decision stream with temporal morphological opening (2) and closing (7). This ablation also produced the raw/residual preprocessing split now used system-wide.

On clean labels config-D and Thermo-X3D are nearly tied in-domain (F_1±2: 76.9% vs. 78.5%), but on the held-out room the network won decisively (73.1% vs. 45.3% for the earlier T5 checkpoint) — the rule's blob-area constants are room-tuned, which conflicts with the no-calibration constraint. Config-D therefore remains a validation baseline; the deployed detector is the network.

5 Runtime

Measured on the development PC (Intel i7-11700K class, 8 cores; single inference, real frames). Pi 5 estimates scale the thread-matched measurements by the per-core gap (Cortex-A76 2.4 GHz vs. ∼4.9 GHz; factor ≈3–3.5×), pending on-device validation.

Model (per frame / per window)	PC CPU	PC (4 threads)	Pi 5 estimate
Tateno preprocessing	0.02 ms	—	∼0.1 ms
FireSVM	1.19 ms	—	3.5–5 ms
MobileNet-SSD	2.6 ms	1.8 ms	8–10 ms
Thermo-X3D (torch fp32, before work)	93.7 ms	—	—
Thermo-X3D (ONNX fp32)	34.5 ms	—	—
Thermo-X3D (ONNX fp32, denormals flushed)	**12.9 ms**	14.7 ms	**45–60 ms (4 cores)**
Thermo-X3D (ONNX fp16, denormals flushed)	13.3 ms	—	Pi candidate (native fp16)
Thermo-X3D (ONNX int8)	60.8 ms	—	untested
Full tick: 3×(pre+fire+person) + contact	≈24 ms		≈65–90 ms
8 Hz budget	125 ms		125 ms

Table 7: CPU inference runtime. The X3D speedup chain is 7.3× total: ONNX export (2.7×, bit-identical F_1), then flushing *denormal weights* — 19.3% of the trained weights were subnormal floats ($\sim 10^{-41}$, a weight-decay artifact) that trigger CPU microcode penalties; zeroing them in the model changed no output bit (max $|\Delta\text{logit}| = 0$) and gave another 2.7×. Future retrains should clamp subnormal weights post-training.

fp16 vs. fp32-flushed — and why the deployed model is fp32-flushed on x86. Converting the model to fp16 also reaches ≈13 ms on the PC, essentially tied with the fp32-flushed model, and it is near-lossless (max $|\Delta p| = 0.0015$ vs. bit-identical for the flush). This tie is the whole point: fp16's apparent speedup on x86 *is* the denormal flush — casting the weights to fp16 underflows those $\sim 10^{-41}$ values to zero as a side effect, and x86 has no native fp16 arithmetic unit (onnxruntime inserts fp16→fp32 casts and computes in fp32 anyway). So on the PC we deploy the fp32-flushed model because it is exactly as fast and provably bit-identical. The Raspberry Pi 5 is the opposite case: its Cortex-A76 *does* have native fp16 SIMD (ARMv8.2 fp16 extension), so there fp16 can compute in half precision and should beat fp32 — making the fp16-flushed checkpoint the leading Pi candidate, to be confirmed on-device. (int8 is slower everywhere on our hardware: onnxruntime's CPU backend has no optimized int8 3-D convolution path, and neither x86 nor the A76 gains from int8 here.)

In the demo application the three per-camera stacks run on parallel worker threads and X3D on a fourth, with depth-1 queues that drop stale frames; the estimated Pi tick (∼65–90 ms) fits the 125 ms budget at 8 Hz with margin. All numbers await confirmation on the Pi itself; the benchmark scripts run there unmodified.

6 Reproducibility

- Benchmarks: `scripts/eval_waveshare_fire.py`, `eval_waveshare_human.py`, `eval_x3d_backends.py`, `benchmark_inference_cpu.py`, `profile_fire_svm.py`.
- Result files (`reports/`): `waveshare_fire_results.json`, `waveshare_human_results.json`, `x3d_T5_v2_results.json`, `x3d_backend_comparison.json`, `cpu_inference_benchmark.json`, `x3d_onnx_runtime.json`.
- Deployment: branch `demo-linux` (weights included); live demo `python -m demo.app -live`.